

دانشگاه صنعتی شریف
دانشکده‌ی مهندسی کامپیوتر

عنوان:

تمرین عملی نخست، پرسش ۲

اعضای گروه

سجاد فتحیان — ۴۰۳۱۰۸۰۹۳
رهام قاسمی — ۴۰۳۱۰۶۴۵۴

نام درس

درس سیستم‌های عامل
Systems Operating

۱-۱۴۰۵

نام استاد درس

دکتر حسین اسدی

فهرست مطالب

۱ نصب ۳

۲ تصویر و کانتینر ۴

۳ مدیریت کانتینر ۵

۳ـآ سوال ۵

۴ مقایسه Bare Metal، ماشین مجازی و داکر ۷

۴ـآ CPU Intensive ۸

۴ـب Memory Intensive ۹

۴ـپ IO Intensive ۱۰

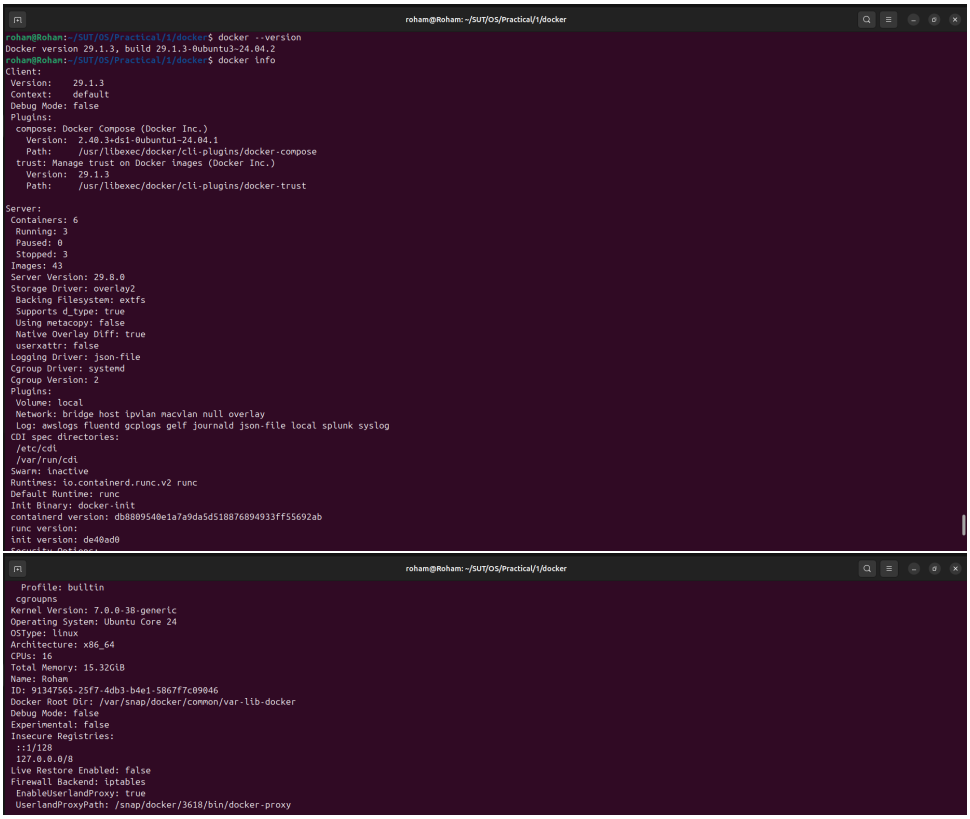
فهرست تصاویر

۱ بررسی نسخه ۳
۲ اجرای برنامه hello-world ۳
۳ اجرای تصویر اوبونتو ۴
۴ بررسی وضعیت کانتینرها و حذف آنان ۵
۵ مقایسه با بنچمارک CPU Intensive ۸
۶ مقایسه با بنچمارک Memory Intensive ۱۰
۷ مقایسه با بنچمارک IO Intensive ۱۱

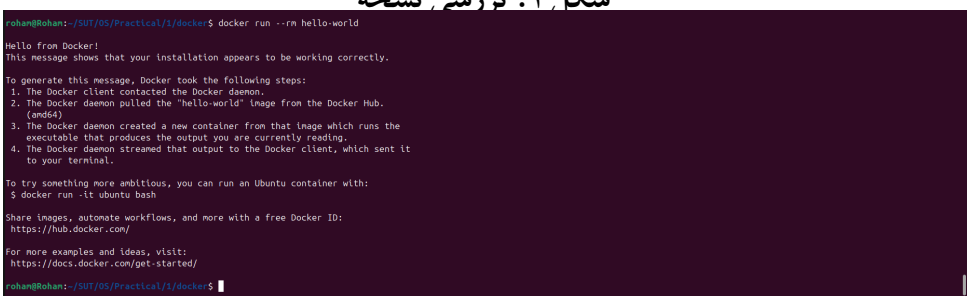
۱ نصب

در این بخش بدون چالش خاصی داکر[۱] بر دستگاه نصب شده و برنامهٔ hello-world اجرا شد.

```
۱ docker --version
۲ docker info
۳ docker run --rm hello-world
```



شکل ۱: بررسی نسخه



شکل ۲: اجرای برنامه hello-world

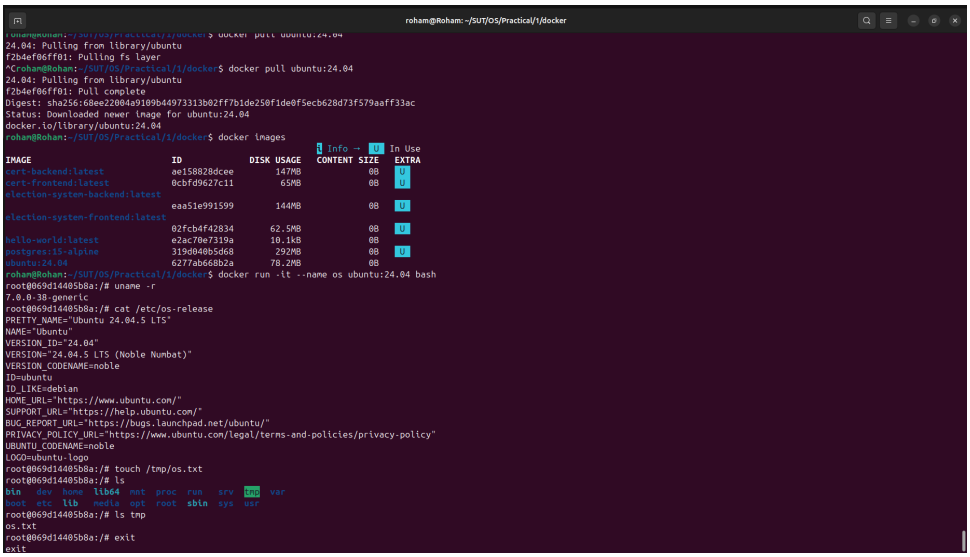
---

[۱] Docker

۲ تصویر و کانتینر

در این بخش یک تصویر[۲] اوبونتو[۳] اجرا شده و چند دستور داخل آن بررسی می‌گردد.

```
docker pull ubuntu:24.04
docker images
docker run -it --name os ubuntu:24.04 bash
uname -r
caat /etc/os-release
touch /tmp/os.txt
exit
```



شکل ۳: اجرای تصویر اوبونتو

در این بخش ابتدا تصویر ubuntu:24.04 واکشی[۴] شده و سپس تصاویر فعلی داکر بررسی می‌شوند؛ قابل توجه است تعدادی تصویر از پروژه‌های شخصی نیز در کنار تصویر اوبونتو قابل ملاحظه می‌باشد. در ادامه تصویر اوبونتو تحت نام "os" در داکر اجرا شده‌است و در ادامه برخی دستورات لینوکس در اوبونتوی داخلی اجرا گردیده است.

در این چند دستور برای تغییر فایل‌ها و بررسی فایل‌های داخل سیستم عامل درونی اجرا شده

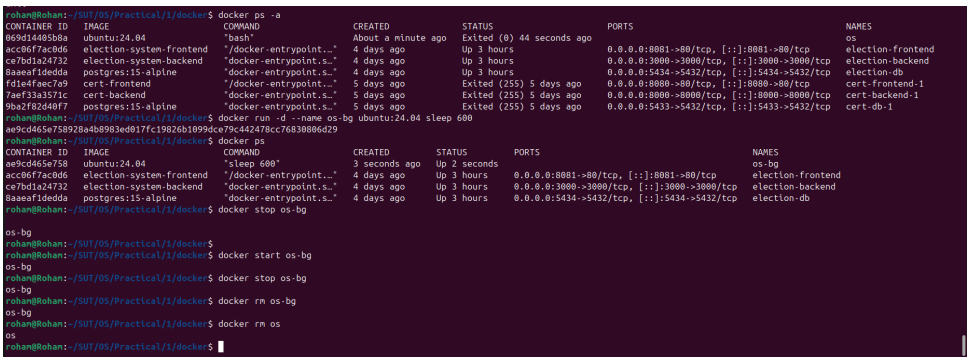
Image[۲]
Ubuntu[۳]
Pull[۴]

است و برای راستی‌آزمایی دستورات `ls` و خروجی آن‌ها قابل مشاهده می‌باشند.

۳ مدیریت کانتینر

در این بخش یک تصویر دیگر از اوبونتو اجرا شده و به یکی از این تصاویر دستور `sleep` داده شده است و در این حین وضعیت کانتینر[۵]ها بررسی می‌گردند و در انتها نیز کانتینرها حذف می‌شوند.

```
docker run d --name os-bg ubuntu:24.04 sleep 600
docker ps
docker stop os-bg
docker start os-bg
docker stop os-bg
docker rm os-bg
docker rm os
```



شکل ۴: بررسی وضعیت کانتینرها و حذف آنان

۳ـآ سوال

تصاویر یک برنامه فقط خواندنی[۶] می‌باشد که شامل برخی کدها و کتابخانه‌ها و تنظیمات پیکربندی[۷] می‌شود در حالی که کانتینر یک نمونهٔ تصویر درحال اجرا می‌باشد که علاوه بر موارد تصویر، یک

[۵] Container
[۶] Read-Only
[۷] Settings Configuration

لایهٔ قابل نوشتن[۸] دارد و به همین دلیل هنگامی که از کانتینر خارج می‌شویم همچنان آن لایهٔ نوشته شده در حافظه ذخیره می‌شود و پس از راه‌اندازی مجدد داده‌ها بازیابی می‌شوند؛ مگر آن که کانتینر کاملا حذف گردد.

---

[۸] Writable

۴ مقایسه Bare Metal، ماشین مجازی و داکر

اطلاعات سیستم Bare Metal

جدول ۱: مشخصات سخت‌افزاری و نرم‌افزاری سیستم میزبان (Bare Metal)

مشخصه	مقدار
مدل پردازنده	13th Gen Intel(R) Core(TM) i7-13620H
تعداد رشته‌های پردازشی	16 (NUMA node0: 0–15)
حافظه اصلی	15 GiB
توزیع سیستم‌عامل	Ubuntu 24.04.3 LTS (Noble)
نسخه هسته	7.0.0-38-generic

اطلاعات Docker

جدول ۲: مشخصات (Docer)

مشخصه	مقدار
نسخه	Docker version 29.1.3, build 29.1.3-0ubuntu3 24.04.2
سطخ دسترسی	Unbound

اطلاعات Virutal Machine

جدول ۳: مشخصات ماشین مجازی (VM - Multipass)

مشخصه	مقدار
توزیع سیستم‌عامل	Ubuntu 24.04.5 LTS
تعداد پردازنده‌های مجازی	4
حافظه رم تخصیص‌یافته	3.8 GiB (≈ 4 GB)
فضای دیسک	9.6 GiB (≈ 10 GB)

اطلاعات Sysbench

در تمامی محیط‌ها از `sysbench 1.0.20` استفاده شده است.

۴ـآ CPU Intensive

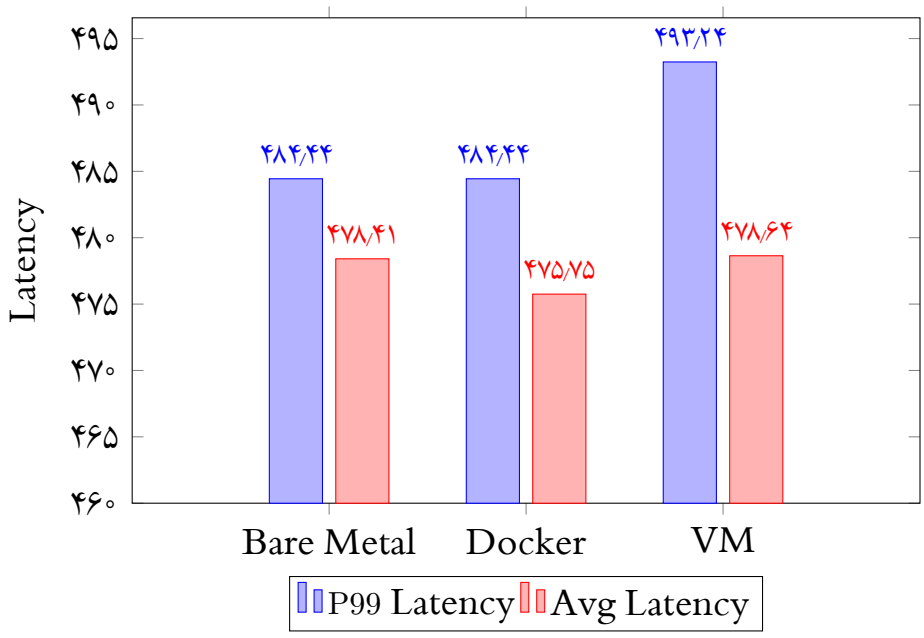
توضیح محک

این محک با استفاده از ۴ ریسمان[۹] تلاش می‌کند، اعداد اول بین ۳ تا ۲۰۰۰۰۰۰ را به روش کلاسیک بیابد، و به ندرت به I/O یا حافظه دسترسی خواهد داشت.

پیش‌بینی

با توجه به اینکه دسترسی به حافظه و I/O ناچیز است، پیش‌بینی می‌شود که هر سه محیط اجرا عملکرد نسبتا مشابهی در ۹۹ صدک برتر داشته باشند، به طور میانگین ممکن است که در ماشین مجازی بدلیل سرباز مجازی سازی ناشی مدیریت وقفه‌های مجازی تاخیر افزایش داشته باشد.

شبیه‌سازی



شکل ۵: مقایسه با بنچمارک CPU Intensive

[۹] Thread

تحلیل نتایج

همانطور که مشاهده می‌شود در ۹۹ صدک، داکر و Bare Metal عملکرد و مشابهی داشته‌اند و ماشین مجازی ضعیف‌تر عمل کرده است که مشابه پیش‌بینی می‌باشد. همچنین در میانگین هم همانطور که پیش‌بینی شده است، ماشین مجازی ضعیف‌ترین عملکرد را داشته است. البته داکر عملکرد خیلی بهتری نسبت به دو محیط دیگر داشته است که از پیش‌بینی ما خارج است؛ به نظر می‌آید که این برتری از معماری داکر مشتق نگشته و ناشی از مینیمالیست بودن داکر و خطاهای اندازه‌گیری در زمان اجرا باشد.

۴ـ ب Memory Intensive

توضیح محک

این مجک یک بافر در حافظه اصلی اختصاص داده و سپس به صورت تصادفی به اندازه ۲۰ گیگابایت داده میذخواند و می‌نویسد. با توجه به اینکه این محک در محاسبه و I/O سربار ندارد، صرفا تاخیر نوشتن را مورد بررسی قرار می‌دهد.

پیش‌بینی

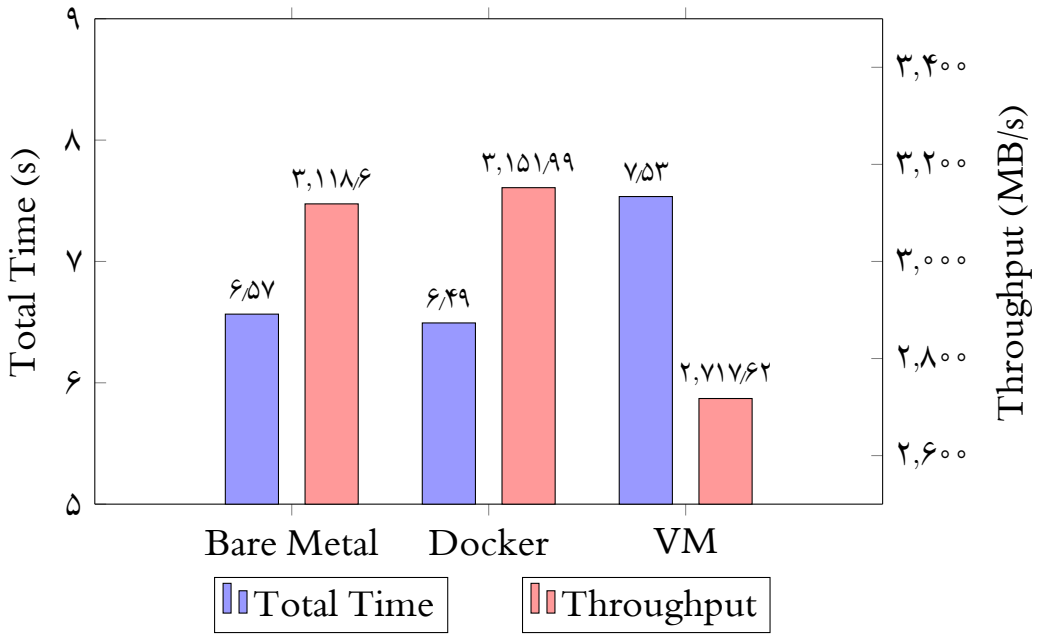
با توجه به این مورد که محیط داکر و Metal Bare مجددا معماری مشابهی دارند، انتظار می‌رود کارایی مشابهی اتفاق بیفتد، لیکن از سمت دیگر با توجه به این در این دو محیط برای ترجمه آدرس حافظه مستقیما از جدول صفحات[۱۰] استفاده می‌کند، نسبت به ماشین مجازی که ابتدا آدرس مجازی میهمان را به آدرس مجازی میزبان تبدیل کرده و سپس سراغ حدول صفحات می‌رود، کارایی بهتری قابل انتظار است.

شبیه‌سازی

تحلیل نتایج

طبق پیش‌بینی عملکرد Bare Metal و Docker به شدت به هم نزدیک هست، در حالی که زمانی کل ماشین مجازی بیشتر بوده و Throughput کمتری ثبت کرده است؛ در واقع کارایی کمتری

[۱۰] Page Table



شکل ۶: مقایسه با بنچمارک Memory Intensive

داشته است.

۴ـپ IO Intensive

توضیح محک

در این محک، ۴ گیگابایت فایل در دیسک ساخته می‌شود و سپس به صورت متوالی دستورات نوشتن بر این فایل‌ها اجرا می‌شود، با توجه به کم بودن کارکرد CPU و تمرکز اصلی بر دیسک به جای حافظهٔ اصلی، این محک عملکرد I/O را بررسی می‌کند.

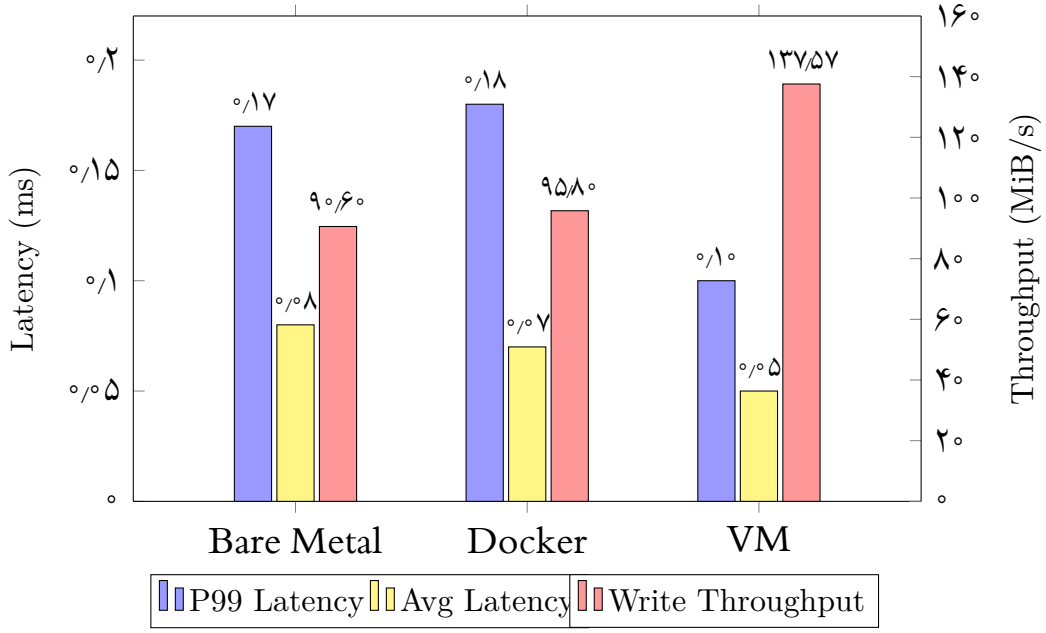
پیش‌بینی

توقع می‌رود که Bare Metal به این دلیل که هیچ لایهٔ میانی و سربار اضافی ندارد و مستقیما به فایل‌سیستم درخواست می‌زند، بهترین عملکرد را داشته باشد (البته این سربار جزئی است و توقع می‌رود تقریبا کارایی مشابهی داشته باشند)، پس از آن Docker به دلیل حضور لایهٔ میانی با سربار جزئی، در رتبهٔ دوم باشد، و سپس ماشین مجازی بدترین عملکرد را خواهد داشت، ماشین مجازی حق استفاده مستقیم از I/O را ندارد و باید درخواست از طریق شبیه‌سازی لایهٔ رابط[۱۱] اتفاق بیفتد

[۱۱] Hyperviser

و به این دلیل به مراتب کندتر است.

شبیه‌سازی



شکل ۷: مقایسه با بنچمارک IO Intensive

تحلیل نتایج

مطابق انتظار داکر و Bare Metal عملکرد مشابهی دارند، لیکن ماشین مجازی علیرغم انتظار عملکرد خیلی بهتری ارائه داده است؛ این احتمالا به این دلیل خواهد بود که درخواست نوشتن در دیسک توسط سیستم عامل میهمان به درستی ارزیابی نمی‌گردد. به طور کلی موقع درخواست نوشتن در دیسک، در سطح سخت‌افزار داده‌ها ابتدا در [۱۲] نوشته شده و سپس به دیسک انتقال داده می‌شوند، هنگامی که بنچمارک در سیستم عامل میزبان اجرا می‌شود درخواست می‌دهد که زمان سیگنال وضعیت پس از نوشتن کامل در دیسک برگردانده شود، در حالی که در سیستم عامل میهمان، پردازه توانایی این را ندارد که حافظهٔ نهان سیستم عامل میزبان را دور بزند و به همین دلیل عملکرد بهتری نسبت به سیستم عامل میهمان نشان خواهد داد؛ به عبارت دیگر در ماشین مجازی همچنان درحالی که داده در دیسک ذخیره نشده است، سیگنال اتمام کار دریافت می‌شود.

[۱۲] Cache Page